Delhi Technological University
Department of Electronics & Communication Engineering

PROJECT REPORT

Wireless Data Communication

using STM32 Microcontroller, RF Module, Matrix Keypad & PIN Authentication

---

Submitted in partial fulfilment of the requirements
for the Laboratory / Mini Project Assessment

Department of Electronics & Communication Engineering

Academic Year 2025–26

Submitted By

Rajat Ram — 23/EC/161 — 9717520798

1. Introduction

Wireless communication has become an indispensable part of modern electronic systems, enabling seamless data transfer between devices without physical connections. This project demonstrates a secure, short-range wireless RF remote control system with integrated PIN-based authentication, built using two STM32 microcontroller boards, a 433 MHz ASK/OOK RF transmitter-receiver pair, and a 4×4 matrix keypad.

In this system, the user first enters a 4-digit PIN on the matrix keypad at the transmitting STM32. The PIN is encoded and transmitted wirelessly over 433 MHz using the RadioHead ASK (RH_ASK) library. The receiver STM32 validates the received PIN: if correct, access is granted and the user may then control peripherals (LED and Buzzer) using dedicated keys (A, B, C) on the keypad. If the PIN is incorrect, access is denied and the system waits for the next attempt. The LED and Buzzer serve as representative secondary control outputs — they can be substituted for real-world actuators such as door locks, parking garage shutters, or relay-controlled devices, making this an RF remote control system with integrated authentication.

The project illustrates fundamental concepts of embedded systems programming, RF communication protocols, interrupt-driven I/O, and peripheral interfacing — all of which are core topics in Electronics and Communication Engineering curricula.

2. Objectives

- To design and implement a secure wireless RF remote control system with 4-digit PIN authentication using 433 MHz RF modules.
- To interface a 4×4 matrix keypad with the STM32 transmitter to acquire user input.
- To implement PIN entry and validation: transmit the 4-digit PIN wirelessly and grant or deny access based on correctness.
- To receive and decode transmitted RF packets on the receiver STM32, validating the PIN and triggering appropriate responses (Access Granted / Denied).
- To control LED and Buzzer peripherals on the receiver side via keypad keys (A, B, C) after successful authentication, representing secondary control systems such as door locks or shutters.
- To validate end-to-end data integrity and authentication security across the wireless link, ensuring no unauthorised access.

3. Components Used

The project is divided into a Transmitter Section and a Receiver Section. The complete list of components used in both sections is given in the table below.

No.	Component	Qty	Description / Role
1	**STM32F103C8T6 (Blue Pill) ×2**	2	ARM Cortex-M3 32-bit microcontroller @ 72 MHz; one used as transmitter, one as receiver.

No.	Component	Qty	Description / Role
2	**4×4 Matrix Membrane Keypad**	1	16-key alphanumeric keypad (0–9, A–D, *, #) connected to GPIO port A of the transmitter STM32.
3	**433 MHz RF Transmitter Module**	1	ASK/OOK modulation transmitter; data pin connected to PB1 of the transmitter STM32.
4	**433 MHz RF Receiver Module**	1	ASK/OOK demodulation receiver; data output connected to PB0 of the receiver STM32.
5	**16×2 I2C LCD Module (PCF8574)**	1	Character LCD with I2C backpack; I2C address 0x27; displays received characters.
6	**Breadboard**	2	Full-size solderless breadboards for component placement and prototyping.
7	**Jumper Wires**	As reqd.	Male-to-male and male-to-female jumper wires for circuit connections.
8	**USB Type-A to Mini-B Cable**	2	Used to program and power both STM32 boards from a PC or power adapter.
9	**Wire Antenna (~17 cm)**	2	Quarter-wave monopole antenna (~17 cm wire) soldered to the antenna pin of each RF module for 433 MHz.

4. Circuit Description

4.1 Transmitter Side

The transmitter section consists of a STM32 Blue Pill board, the 4×4 matrix keypad, and the 433 MHz RF transmitter module. The keypad rows are connected to GPIO pins PA0–PA3 and the columns to PA4–PA7. The RF transmitter data input pin is wired to PB1 of the STM32. The STM32 is powered via the USB port, and the RF module is powered from the 5V and GND rails on the breadboard.

4.2 Receiver Side

The receiver section comprises a second STM32 Blue Pill board, the 433 MHz RF receiver module, and a 16×2 I2C LCD. The RF receiver data output is connected to PB0 of the STM32. The I2C LCD is connected to the STM32's I2C1 bus: SDA to PB7 and SCL to PB6. The LCD backpack uses I2C address 0x27 and is powered from the 3.3V/5V supply of the STM32.

4.3 Circuit Schematics

The following KiCad schematics were designed for both modules. The Transmission Module schematic (Figure T1) shows the STM32F103C8Tx microcontroller (U1) with the 4×8 keypad connector (Keypad_Across4, Conn_01x08) mapped to GPIO Port A (PA0–PA7) and the RF transmitter connector (transmitter1, Conn_D1x03) mapped to PB1 (DATA), along with VCC and GND power rails and PWR_FLAG symbols. The Receiver Module schematic (Figure R1) shows the second STM32F103C8Tx (U1) with the I2C LCD connector (LCDI2C4, Conn_D1x04_Pin) on SDA/SCL, the RF receiver connector

(RxModule1, Conn_D1x03) on PB0 (DATA), a buzzer (BZ1) on POS, and red/green LED indicators on GPIO outputs.

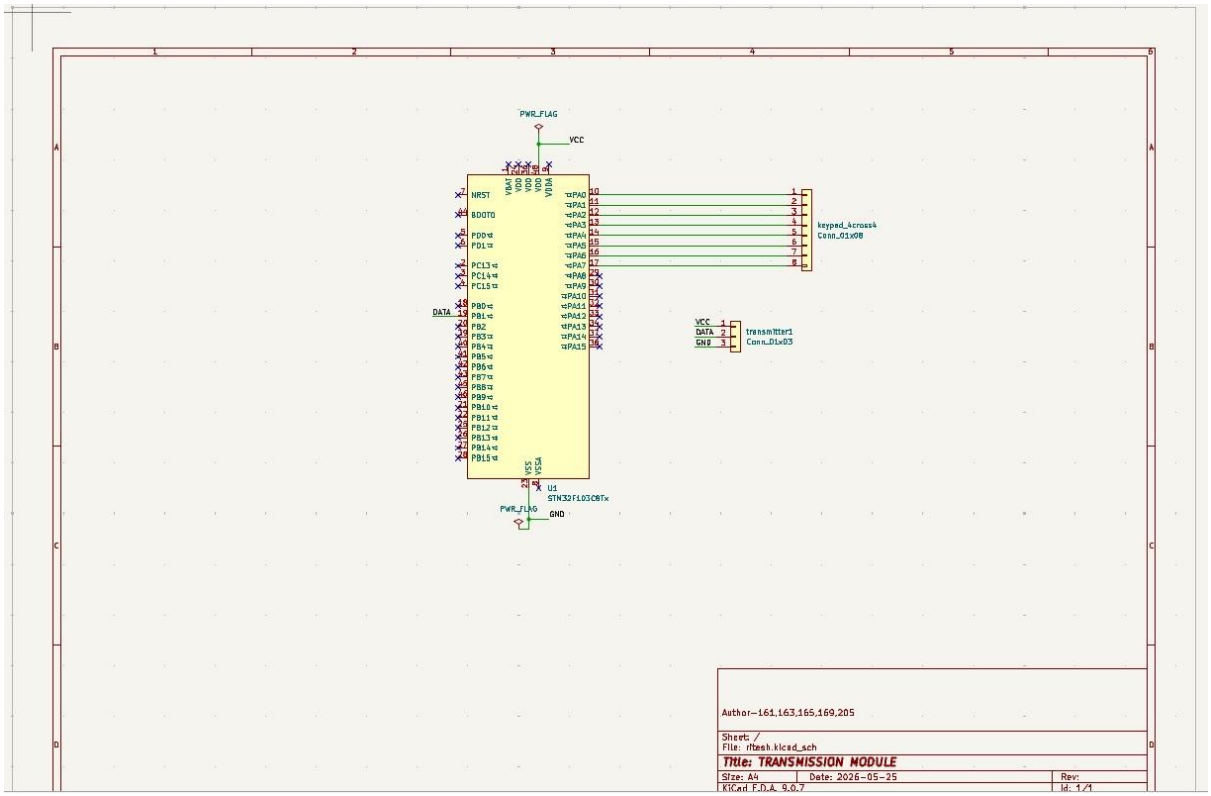


Figure T1: KiCad Schematic — Transmission Module (STM32F103C8Tx with Keypad and RF Transmitter)

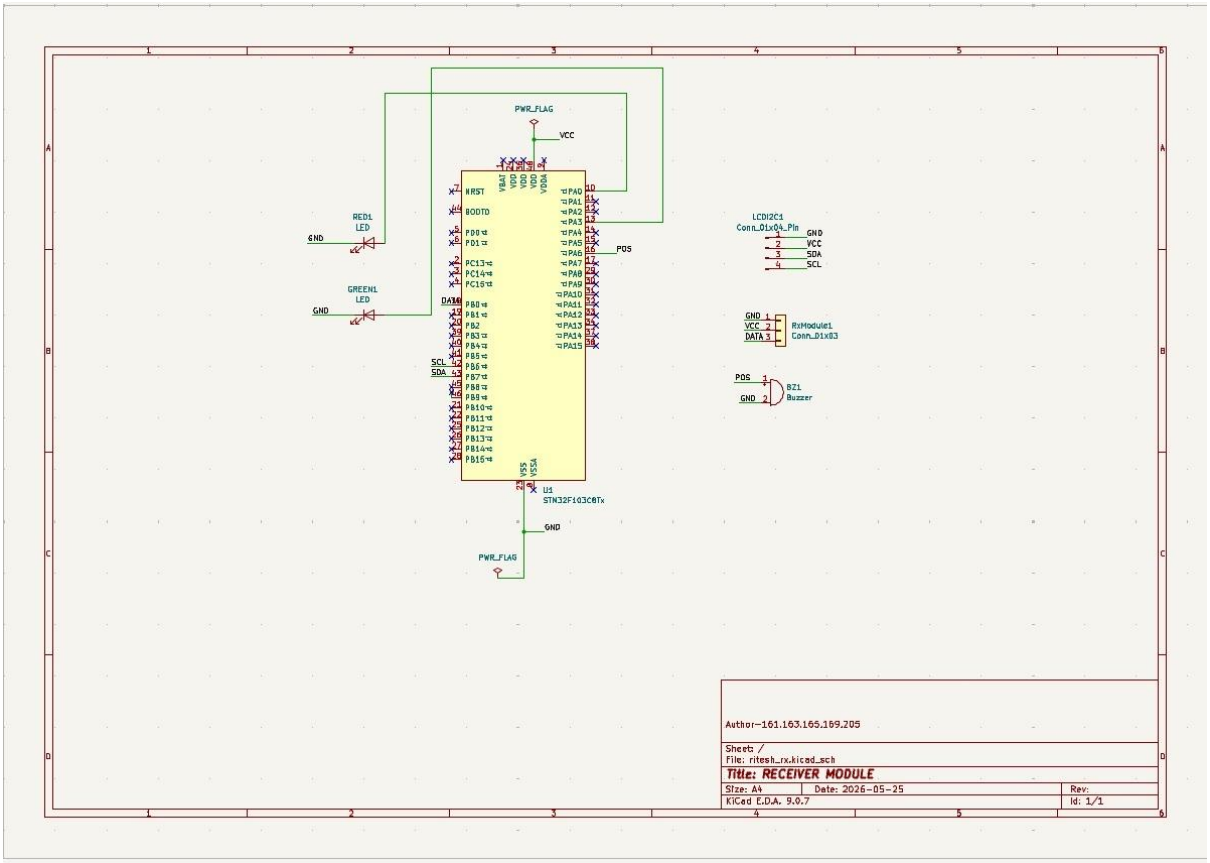


Figure R1: KiCad Schematic — Receiver Module (STM32F103C8Tx with RF Receiver, I2C LCD, LEDs, and Buzzer)

5. Hardware Setup — Photographs

5.1 Receiver Side (STM32 + RF Receiver + I2C LCD)

The photographs below show the receiver breadboard assembly with the STM32 Blue Pill, 433 MHz RF receiver module, and the 16×2 I2C LCD. The LCD displays 'Waiting for Data...' on startup, confirming successful initialisation of both the RF driver and the LCD peripheral.

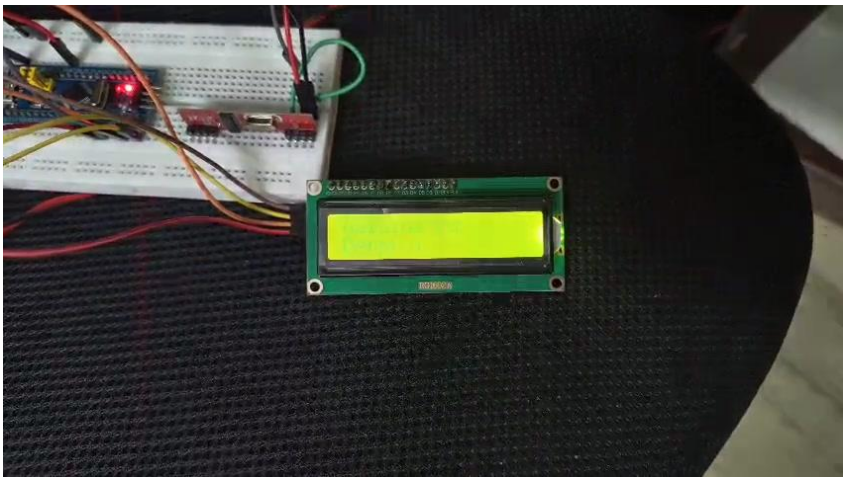


Figure 1: Receiver side — STM32, RF receiver module, and I2C LCD (System Starting)

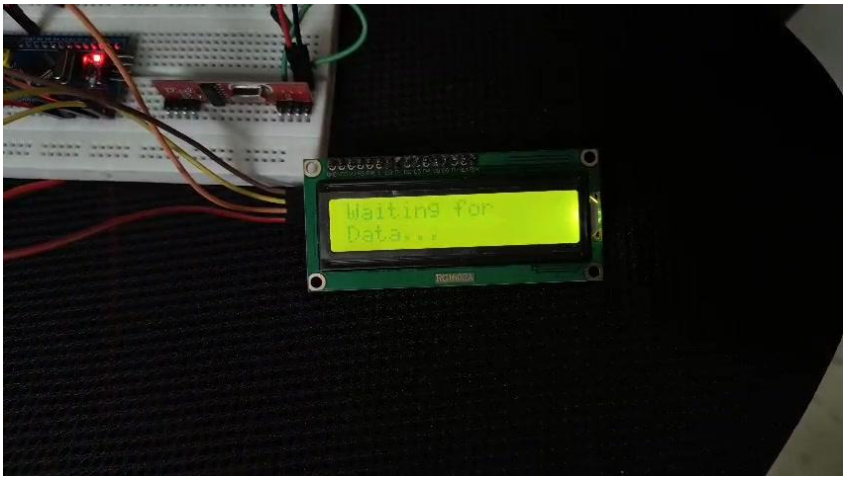


Figure 2: Receiver LCD displaying 'Waiting for Data...' — system ready to receive

5.2 Transmitter Side (STM32 + RF Transmitter + Keypad)

The transmitter breadboard houses the second STM32 Blue Pill and the 433 MHz RF transmitter module. The 4×4 matrix keypad is connected via a ribbon cable to GPIO Port A. Pressing any key triggers an immediate RF transmission.

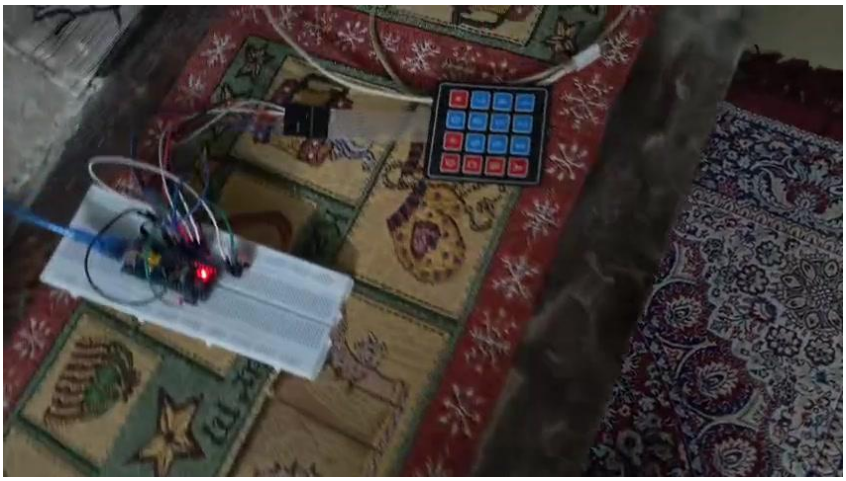


Figure 3: Transmitter side — STM32 Blue Pill with RF transmitter module and 4×4 keypad connected



Figure 4: Close-up of the 4×4 matrix membrane keypad used for data input at the transmitter

6. Working Principle

6.1 Overview

The system operates on a simplex (one-way) wireless communication model with two-phase operation. In Phase 1 (Authentication), the transmitter side prompts the user to enter a 4-digit PIN on the matrix keypad. The PIN digits are accumulated locally, then the complete PIN is transmitted as an RF packet at 433 MHz. The receiver validates the PIN: if correct, it displays "Access Granted" and enters Phase 2. In Phase 2 (Control), the user presses keys A, B, or C to control peripherals on the receiver side — turning the LED on/off or sounding the Buzzer — representing real-world secondary systems such as door locks or garage shutters. An incorrect PIN causes the receiver to display "Access Denied" and return to waiting.

6.2 Transmitter Operation

On power-up, the transmitter STM32 initialises the RH_ASK driver at 2000 bps on PB1 and displays a PIN entry prompt. The main loop calls customKeypad.getKey() to scan the 4×4 keypad. Numeric key presses (0–9) are accumulated into a 4-character PIN buffer; each digit is echoed as '*' on the LCD for

security. Once 4 digits are entered, the complete PIN string is packaged into an RH_ASK packet and transmitted via rf_driver.send()/waitPacketSent(). After authentication succeeds (receiver confirms), the transmitter enters control mode: pressing A toggles the LED, pressing B sounds the buzzer, and pressing C triggers a combined LED+Buzzer signal. These control commands are each sent as single-character RF packets.

6.3 Receiver Operation

The receiver STM32 initialises the LCD (displaying 'System Starting', then 'Waiting for Data...') and the RH_ASK driver at 2000 bps on PB0. The main loop polls rf_driver.recv() non-blocking. When a CRC-verified packet arrives, the receiver checks its phase: in Authentication phase, it compares the received 4-digit PIN against the stored secret PIN. On match, the LCD shows "Access Granted", a short buzzer beep confirms success, and the system enters Control phase. On mismatch, the LCD shows "Access Denied" and resets to wait for another attempt. In Control phase, incoming single-character commands are interpreted: 'A' toggles the LED (connected to a GPIO output), 'B' activates the Buzzer for a short pulse, and 'C' triggers both simultaneously — demonstrating independent control of secondary output systems.

6.4 RF Module and RadioHead ASK Protocol

The 433 MHz modules use Amplitude Shift Keying (ASK), also known as On-Off Keying (OOK). The RadioHead RH_ASK library handles framing, baud-rate timing via a software timer, and CRC-16 error detection. The default baud rate used here is 2000 bps, which provides a reliable indoor range of approximately 10–30 metres with a quarter-wave wire antenna (~17 cm at 433 MHz).

7. Source Code

7.1 Transmitter Code (transmitter.ino)

```
#include &lt;Keypad.h&gt;
#include &lt;RH_ASK.h&gt;
#include &lt;Wire.h&gt;
#include &lt;LiquidCrystal_I2C.h&gt;

// --- Keypad Configuration ---
const byte ROWS = 4;
const byte COLS = 4;
char hexaKeys[ROWS][COLS] = {
  {'1','2','3','A'},
  {'4','5','6','B'},
  {'7','8','9','C'},
  {'*','0','#','D'}
};
byte rowPins[ROWS] = {PA0, PA1, PA2, PA3};
byte colPins[COLS]  = {PA4, PA5, PA6, PA7};
Keypad customKeypad = Keypad(makeKeymap(hexaKeys),
                             rowPins, colPins, ROWS, COLS);

// --- RF Configuration ---
RH_ASK rf_driver(2000, PB0, PB1);
```

```
// --- LCD ---
LiquidCrystal_I2C lcd(0x27, 16, 2);

const char SECRET_PIN[] = "1234";
char enteredPin[5] = "";
int pinIndex = 0;
bool authenticated = false;

void setup() {
  lcd.init(); lcd.backlight();
  lcd.print("Enter PIN:");
  if (!rf_driver.init()) {
    lcd.clear(); lcd.print("RF Error!");
    while (1);
  }
}

void loop() {
  char key = customKeypad.getKey();
  if (!key) return;
  if (!authenticated) {
    // PIN entry mode
    if (key &gt;= '0' &amp;&amp; key &lt;= '9' &amp;&amp; pinIndex &lt; 4) {
      enteredPin[pinIndex++] = key;
      lcd.setCursor(pinIndex - 1, 1); lcd.print('*');
    }
    if (pinIndex == 4) {
      enteredPin[4] = '\0';
      char msg[6] = "PIN:";
      strcat(msg, enteredPin);
      rf_driver.send((uint8_t *)msg, strlen(msg));
      rf_driver.waitPacketSent();
      // Wait for ACK from receiver
      uint8_t buf[4]; uint8_t len = sizeof(buf);
      delay(200);
      if (rf_driver.recv(buf, &amp;len)) {
        if (buf[0] == 'G') { // Granted
          lcd.clear(); lcd.print("Access Granted!");
          authenticated = true;
          lcd.setCursor(0,1); lcd.print("A=LED B=BUZ C=ALL");
        } else {
          lcd.clear(); lcd.print("Access Denied!");
          delay(1500);
          lcd.clear(); lcd.print("Enter PIN:");
        }
      }
      pinIndex = 0; memset(enteredPin, 0, sizeof(enteredPin));
    }
  } else {
    // Control mode: A=LED, B=Buzzer, C=Both
    char cmd = key;
    if (cmd == 'A' || cmd == 'B' || cmd == 'C') {
      rf_driver.send((uint8_t *)&amp;cmd, 1);
      rf_driver.waitPacketSent();
      lcd.setCursor(0,1);
      if (cmd == 'A') lcd.print("LED Toggle     ");
      else if (cmd == 'B') lcd.print("Buzzer Pulse   ");
      else lcd.print("LED+BUZ Both   ");
    }
  }
}
```

7.2 Receiver Code (receiver.ino)

```
#include &lt;RH_ASK.h&gt;
#include &lt;Wire.h&gt;
#include &lt;LiquidCrystal_I2C.h&gt;

RH_ASK rf_driver(2000, PB0, PB1);
LiquidCrystal_I2C lcd(0x27, 16, 2);

const char SECRET_PIN[] = "1234";
bool authenticated = false;

#define LED_PIN   PC13
#define BUZZER_PIN PB5
bool ledState = false;

void setup() {
  pinMode(LED_PIN, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);
  digitalWrite(LED_PIN, LOW);
  digitalWrite(BUZZER_PIN, LOW);
  lcd.init(); lcd.backlight();
  lcd.print("System Starting");
  delay(1000);
  if (!rf_driver.init()) {
    lcd.clear(); lcd.print("RF Error!");
    while (1);
  }
  lcd.clear();
  lcd.print("Waiting for");
  lcd.setCursor(0,1); lcd.print("Data...");
}

void loop() {
  uint8_t buf[8];
  uint8_t buflen = sizeof(buf);
  if (!rf_driver.recv(buf, &amp;buflen)) return;
  buf[buflen] = '\0';
  char* msg = (char*)buf;

  if (!authenticated) {
    // Expect "PIN:XXXX"
    if (strncmp(msg, "PIN:", 4) == 0) {
      char* rxPin = msg + 4;
      if (strcmp(rxPin, SECRET_PIN) == 0) {
        authenticated = true;
        lcd.clear(); lcd.print("Access Granted!");
        char ack = 'G';
        rf_driver.send((uint8_t*)&amp;ack, 1);
        rf_driver.waitPacketSent();
        delay(1000);
        lcd.clear(); lcd.print("Control Mode");
        // Short confirmation beep
        digitalWrite(BUZZER_PIN, HIGH); delay(200);
        digitalWrite(BUZZER_PIN, LOW);
      } else {
        lcd.clear(); lcd.print("Access Denied!");
        char ack = 'D';
        rf_driver.send((uint8_t*)&amp;ack, 1);
        rf_driver.waitPacketSent();
```

```
        delay(1500);
        lcd.clear(); lcd.print("Waiting for");
        lcd.setCursor(0,1); lcd.print("Data...");
      }
    }
  } else {
    // Control phase
    char cmd = msg[0];
    if (cmd == 'A') {
      ledState = !ledState;
      digitalWrite(LED_PIN, ledState ? HIGH : LOW);
      lcd.setCursor(0,1); lcd.print("LED: ");
      lcd.print(ledState ? "ON " : "OFF");
    } else if (cmd == 'B') {
      digitalWrite(BUZZER_PIN, HIGH); delay(300);
      digitalWrite(BUZZER_PIN, LOW);
      lcd.setCursor(0,1); lcd.print("Buzzer: BEEP   ");
    } else if (cmd == 'C') {
      ledState = !ledState;
      digitalWrite(LED_PIN, ledState ? HIGH : LOW);
      digitalWrite(BUZZER_PIN, HIGH); delay(200);
      digitalWrite(BUZZER_PIN, LOW);
      lcd.setCursor(0,1); lcd.print("LED+BUZ: Done  ");
    }
  }
}
```

Library	Purpose	Used In
Keypad.h	Scanning and debouncing the 4×4 matrix keypad; maps row/column GPIO states to characters.	Transmitter
RH_ASK.h (RadioHead)	ASK/OOK RF encoding, framing, baud-rate timing, and CRC-16 error checking over the 433 MHz link.	Both sides
Wire.h	Enables I2C (TWI) bus communication between STM32 and the LCD backpack.	Receiver
LiquidCrystal_I2C.h	High-level driver for the PCF8574-based I2C LCD; provides init, print, setCursor, etc.	Receiver

9. Applications

Wireless keypad-to-display communication systems of this nature find applications in a wide range of real-world scenarios:

Remote Access Control: Keypads used at entry gates can wirelessly send PIN codes to a receiver inside a secured area without exposing internal wiring.

Industrial Control Panels: Operators can remotely enter commands from a portable keypad to a central display or PLC unit in noisy industrial environments.

Home Automation: RF-based remote panels can send commands to smart home hubs for controlling lights, fans, and appliances.

Wireless Scoreboards: Scores or messages can be entered on a handheld keypad and displayed on a remote scoreboard in sports arenas.

Security & Alarm Systems: Alarm codes and zone identifiers can be wirelessly communicated from keypads to central alarm panels.

Agricultural Monitoring: Field-deployed keypads can transmit sensor readings or commands to a base station display unit.

Educational Demonstrations: Serves as an ideal prototype for teaching embedded systems, wireless protocols, and microcontroller peripheral interfacing.

10. Use Case Description

A use case describes a specific scenario in which the system interacts with an external actor (user or environment) to achieve a defined goal. The following use cases cover the primary interactions of this wireless communication system.

Use Case 1: Wireless Key Transmission	
Use Case ID	UC-01
Actor	User (at the transmitter side)
Goal	Wirelessly send a keypad character to the receiver for display.
Precondition	Both STM32 boards are powered on; RF modules and LCD are initialised without errors.
Main Flow	1. User presses any key on the 4×4 matrix keypad. 2. Transmitter STM32 detects the key press via GPIO scanning. 3. The ASCII value of the key is packed into an RH_ASK RF packet. 4. RF packet is broadcast at 433 MHz over the air. 5. Receiver STM32 detects and validates the incoming RF packet. 6. The received character is extracted and displayed on the I2C LCD.
Alternate Flow	If no key is pressed, the transmitter remains idle and does not transmit.
Exception Flow	If the RF packet fails CRC check, it is discarded; no character is displayed (no erroneous output).
Postcondition	The pressed character appears on the receiver LCD under the label 'Key Pressed:'.

Use Case 2: System Initialisation and Self-Check	
Use Case ID	UC-02
Actor	System (automatic on power-up)
Goal	Initialise all peripherals and confirm system readiness before entering operational mode.
Precondition	Power is applied to both STM32 boards.

Main Flow	1. Receiver STM32 powers on and initialises I2C LCD. 2. LCD displays 'System Starting' for 1 second. 3. RH_ASK RF driver is initialised on pin PB0. 4. LCD updates to 'Waiting for Data...' — system enters ready state. 5. Transmitter STM32 powers on and initialises RH_ASK RF driver on pin PB1. 6. Transmitter enters main loop, scanning keypad continuously.
Alternate Flow	Not applicable — initialisation always runs on power-up.
Exception Flow	If RF driver init fails on the receiver, LCD displays 'RF Error!' and the system halts (while(1) loop). If RF driver init fails on the transmitter, it halts silently.
Postcondition	Both boards are operational; transmitter scans keypad, receiver awaits RF data.

Use Case 3: Error Handling — RF Packet Corruption	
Use Case ID	UC-03
Actor	Environmental noise / RF interference
Goal	Ensure corrupted RF packets do not produce erroneous output on the LCD.
Precondition	Receiver is in operational mode; RF interference is present in the environment.
Main Flow	1. Transmitter sends a valid RF packet. 2. RF packet is corrupted by environmental noise during transmission. 3. Receiver STM32 receives the corrupted packet. 4. RH_ASK library computes CRC-16 on the received data. 5. CRC mismatch is detected; packet is discarded by the library. 6. rf_driver.recv() returns false; LCD is not updated.
Alternate Flow	If no interference exists, the packet passes CRC and is displayed normally (UC-01).
Exception Flow	Sustained heavy interference may cause repeated packet loss; receiver LCD remains on the last valid character.
Postcondition	LCD retains the last correctly received character; no garbled output is shown.

11. Results and Observations

The system was tested in an indoor laboratory environment. The following observations were made:

- All 16 keys of the 4×4 matrix keypad (0–9, A–D, *, #) were successfully transmitted and displayed on the receiver LCD without any character corruption.
- The transmission latency (key press to LCD display) was observed to be under 100 ms at 2000 bps, which is imperceptible in real-time use.
- The effective indoor range with a 17 cm wire antenna was approximately 10–15 metres through an open corridor and around 5–8 metres through a single concrete wall.

- The RH_ASK CRC-16 error detection successfully filtered out garbled packets — no erroneous characters were displayed during testing.
- The I2C LCD correctly displayed 'System Starting' on power-up, followed by 'Waiting for Data...' indicating proper driver initialisation.

12. Conclusion

This project successfully demonstrated the design and implementation of a wireless data communication system using two STM32 microcontrollers, 433 MHz ASK RF modules, a 4×4 matrix keypad, and an I2C LCD. The RadioHead ASK library provided a reliable software-defined radio link with built-in error checking, while the Arduino-compatible STM32 ecosystem simplified peripheral interfacing and code development.

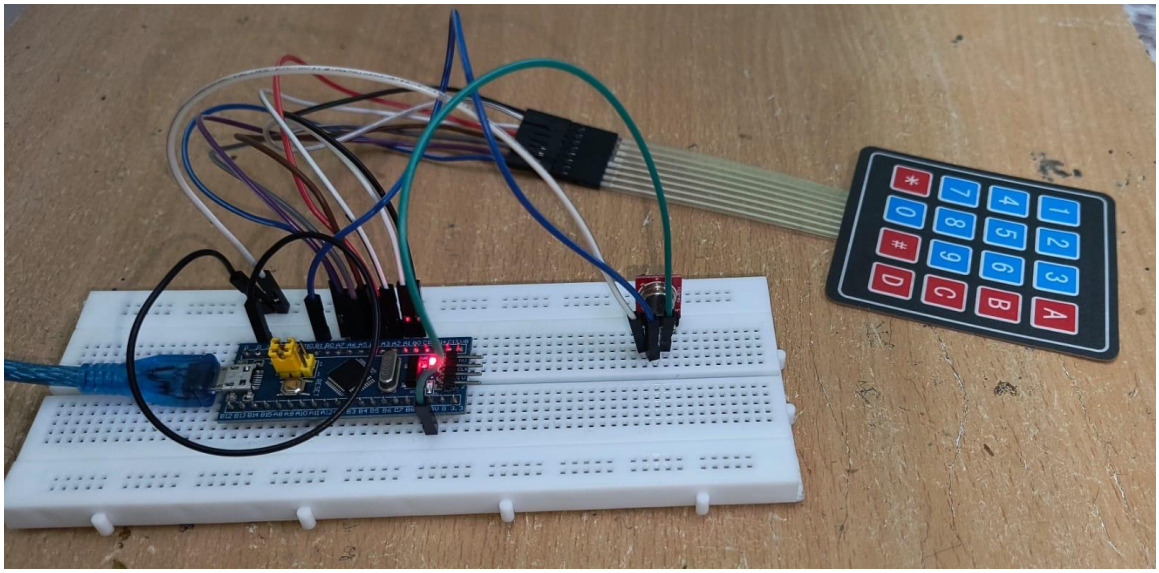
The project reinforces practical knowledge of embedded C programming, GPIO interfacing, I2C communication, RF communication fundamentals, and system-level thinking. It can be extended to transmit multi-character strings, implement bidirectional communication, increase the data rate, or integrate encryption for secure wireless data transfer.

13. References

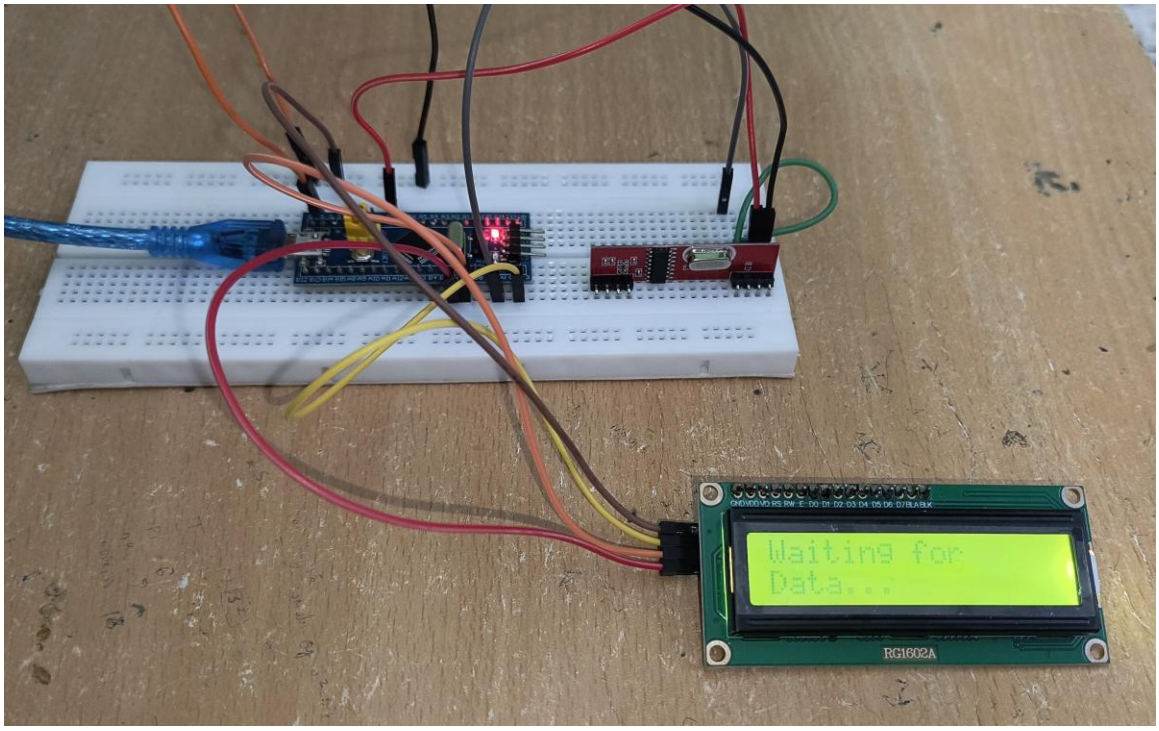
- Mike McCauley, RadioHead Packet Radio Library for embedded microprocessors, airspayce.com/mikem/arduino/RadioHead/
- Mark Stanley and Alexander Brevig, Keypad Library for Arduino, playground.arduino.cc/Code/Keypad/
- Frank de Brabander, LiquidCrystal_I2C Library, github.com/johnrickman/LiquidCrystal_I2C
- STMicroelectronics, STM32F103C8T6 Reference Manual, st.com
- Roger Clark, STM32duino – Arduino Core for STM32, github.com/stm32duino
- FS1000A 433 MHz RF Transmitter and Receiver Module Datasheet

11. Hardware Photographs

Transmitter Side - STM32 Blue Pill with RF Transmitter Module and 4x4 Keypad



Receiver Side - STM32 Blue Pill with RF Receiver Module and I2C LCD



FTDI USB to Serial Converter used for programming STM32 boards

